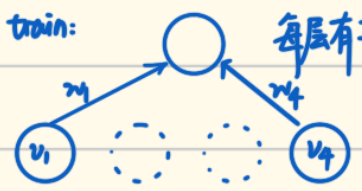
Lecture 7：循环神经网络. RNN. Recurrent Neural Networks

上节课的问题.

1. Dropout

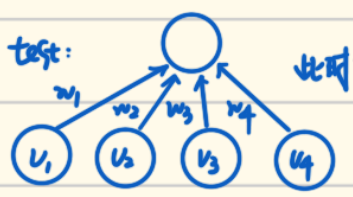
train:



每层有平均 np 个节点被删

$w_1v_1 + w_4v_4$

test:



此时不再删除点. 放输入的总值与之前的比为 $\frac{n}{n-np} = \frac{1}{1-p}$.

$w_1v_1 + w_2v_2 + w_3v_3 + w_4v_4$

2. Normalization.

Normalization 有助于缓解初始化不正确的问题.

但优秀的初始化依然能有更好表现.

非递归.

Recap: Training Non-Recurrent Neural Networks.

① One time setup: 激活函数(activation functions). 预处理(preprocessing). 权重初始化(weight initialization)
一次性设置 标准化(normalization). 迁移学习(transfer learning)

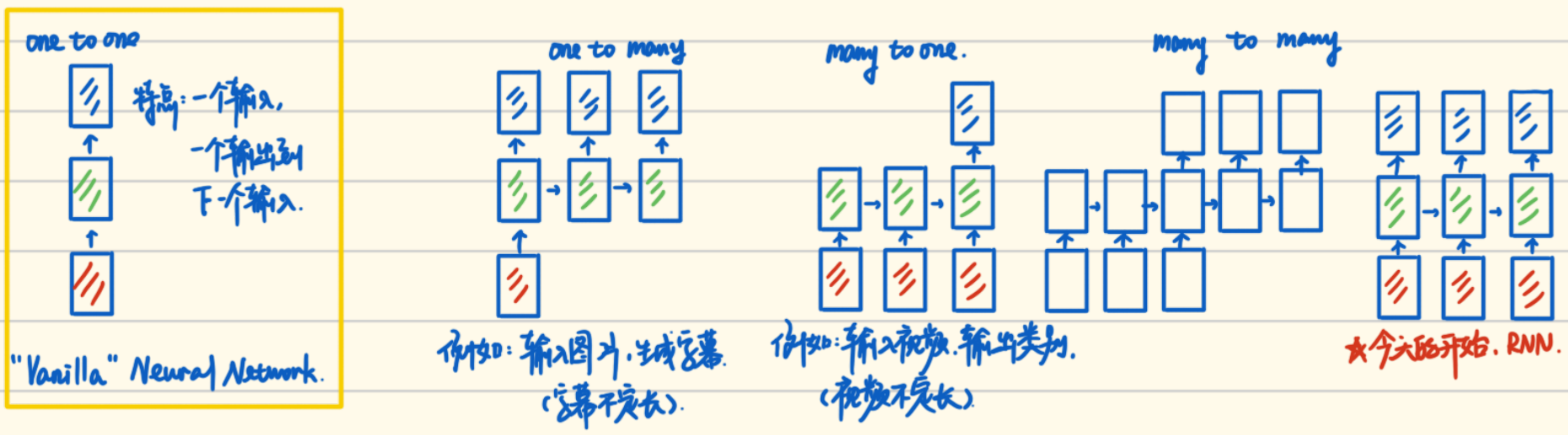
② Training dynamics: 跟踪学习过程, 参数更新, 超参数优化
训练动态 (选择合适的learning rate)

③ Evaluation 评估: 正确率, 运行时间

Today's Lecture:

1. Sequence modeling 序列建模, 非定长输入的模型
2. Transformers 前的一些模型. (RNN及变体).
3. 状态空间模型. (Mamba.)

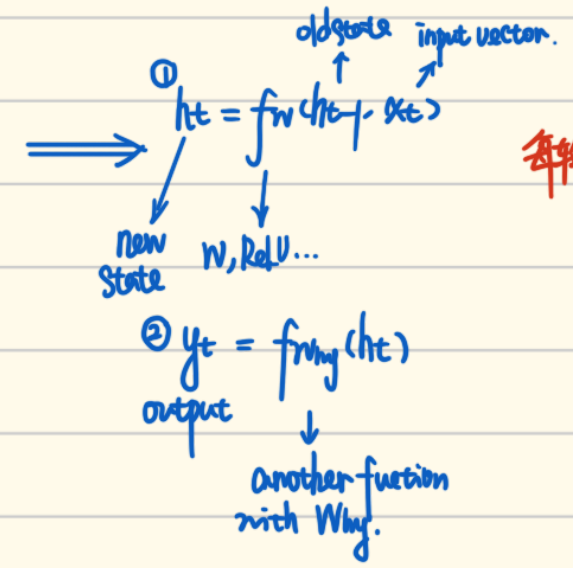
(一). Recurrent Neural Networks



RNN: Recurrent Neural Network.



RNN的内部状态(隐藏状态)会随序列更新.



每轮迭代使用的函数/参数都相同.
仅 x_t, h_t 有差别.

* 一个用香草神经网络实现的 RNN:

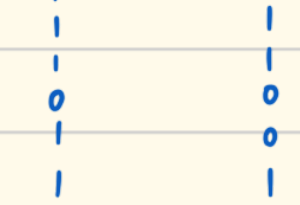
$$\begin{cases} h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t) \\ y_t = W_{hy} h_t \end{cases}$$

tanh: 非线性函数; W_{hh}, W_{xh}, W_{hy}: 三个权重矩阵

Example: 让我们手工搭建一个RNN吧!

想实现的功能:

X: (Sequence) → RNN → Y



当出现当前+上个都是1时Y输出1, 否则0.

$$\begin{cases} h_t = f_W(h_{t-1}, x_t) \\ y_t = f_y(W_{hy} \cdot h_t) \end{cases}$$

希望: h_t 保存当前 x 值, 给下一步传递
y_t 在 h_t 和 x_t 都满足时输出1.

→ $h_t = \begin{pmatrix} \text{Current} \\ \text{Previous} \\ *1 \end{pmatrix}$ $y_t = x_t$ & h_t.Current.

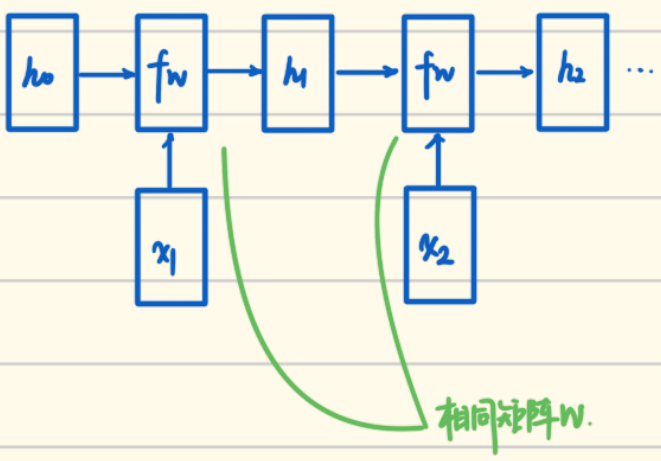
→ $h_0 = (0,0,1)$

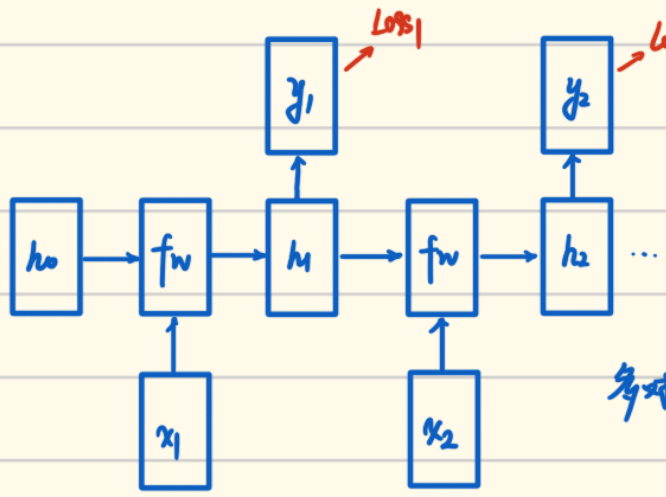
ReLU

$$\begin{cases} h_t = ReLU\left(\begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \cdot h_{t-1} + \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \cdot x_t\right) \\ y_t = ReLU((1,1,-1) \cdot h_t) \end{cases}$$

→ Max(Current + Prevent - 1, 0)

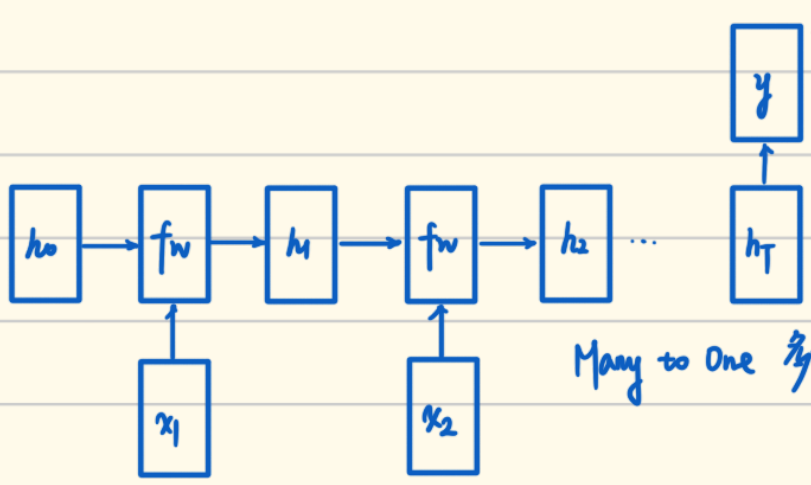
RNN: 如何梯度下降? → 计算图.





整体Loss, 单独Loss 取决于计算方程.
例如: 在实际中计算每个fw的梯度, 相加.

多对多 (Many to Many)

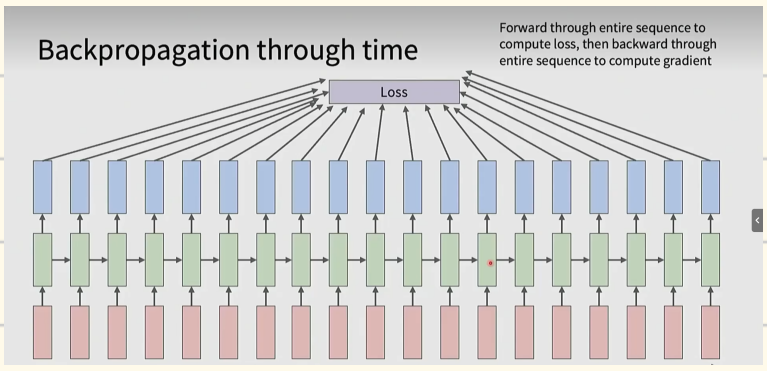


仅依此来反向传播.

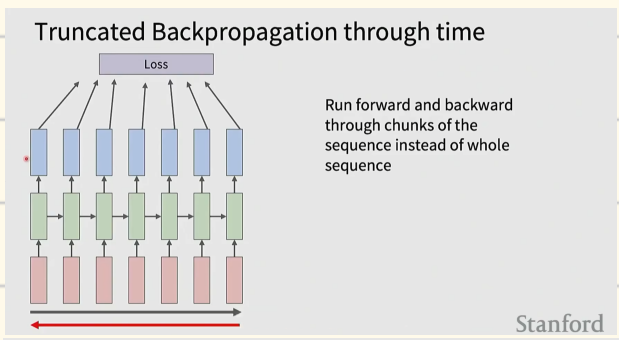
Many to One 多对一.

(backpropagation through time).

面对长序列，整个计算过程会变得很长，占用内存量↑.



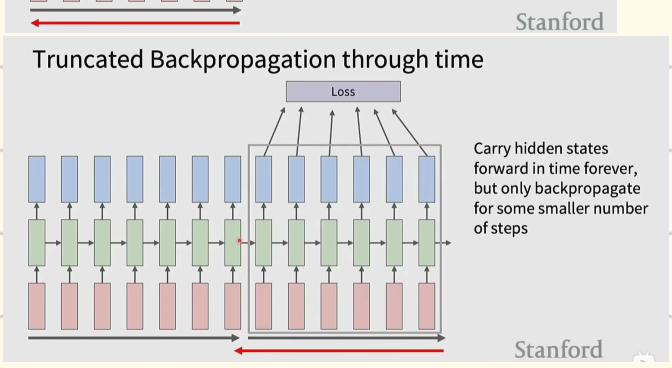
解决方式：Truncated Backpropagation through time.（截断反向传播）.



每一小段窗口，单独计算Loss并获得梯度 → Many to Many，丢失信息

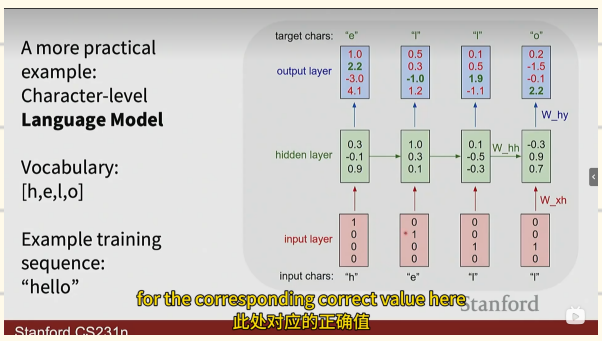
Many to 1：依旧分时计算，但从前向后传梯度.

└→ 并会丢失信息.



A more practical example：Character-level Language Model.

更贴近实际的例子：字符级语言模型.（预测下一输出）.



输入“索引”向量，输出最可能的索引

散入层 ⇒ 将稀疏索引转化为更小、更密的矩阵.

⇒ 前LLM时代的做法，与如今的工具相比在high-level上很相似.

RNN权衡（trade offs）

Advantages：

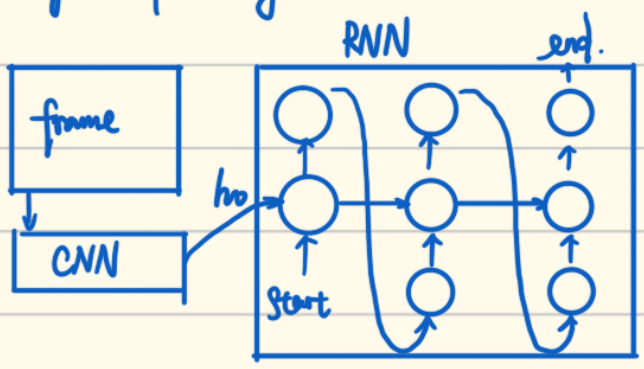
- 任何长度的输入 ← 现代transformer：context length 上下文窗口 / 不固定长的输入
- 在step t的运算，理论上可以使用任何以前步骤的信息.
- 保持等大的模型大小
- 每次timestep都依赖相同的矩阵W，具有良好对称性.

Disadvantages：

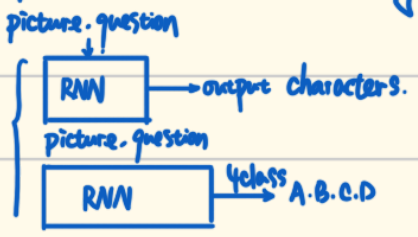
- 递归计算比较慢（training 非常慢）.
- 在实际中，越远的信息越难获得.（定长隐藏状态，随信息增多，会每轮损失一定信息）

RNN in CV.

① Image Captioning



② VQA: Visual Question Answering.



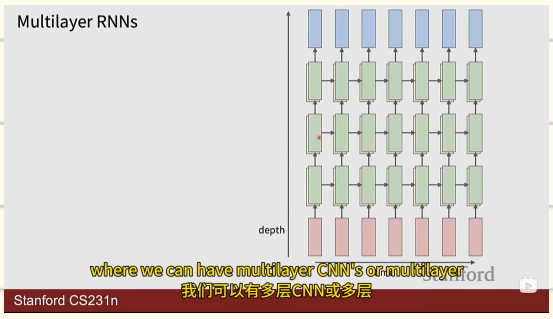
③ Visual Dialog.

④ Visual Language Navigation.

产生错误/误判：相似的、具有迷惑性的动作.

{ 场景、背景的"思维惯性".

Multilayer RNNs 多层RNN.

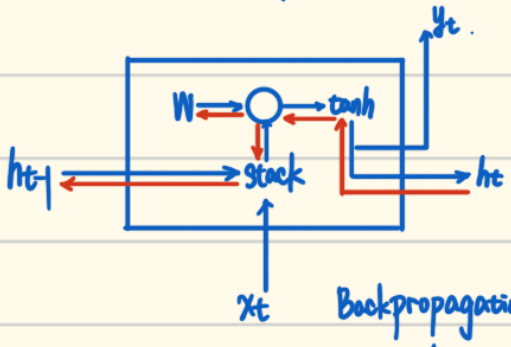


每层都单独处理，所以每层都有自己的隐藏状态.

(复杂、训练效率不高).

RNN Variants: Long Short Term Memory (LSTM). 选择性信息传递.

(RNN变体：LSTMs)

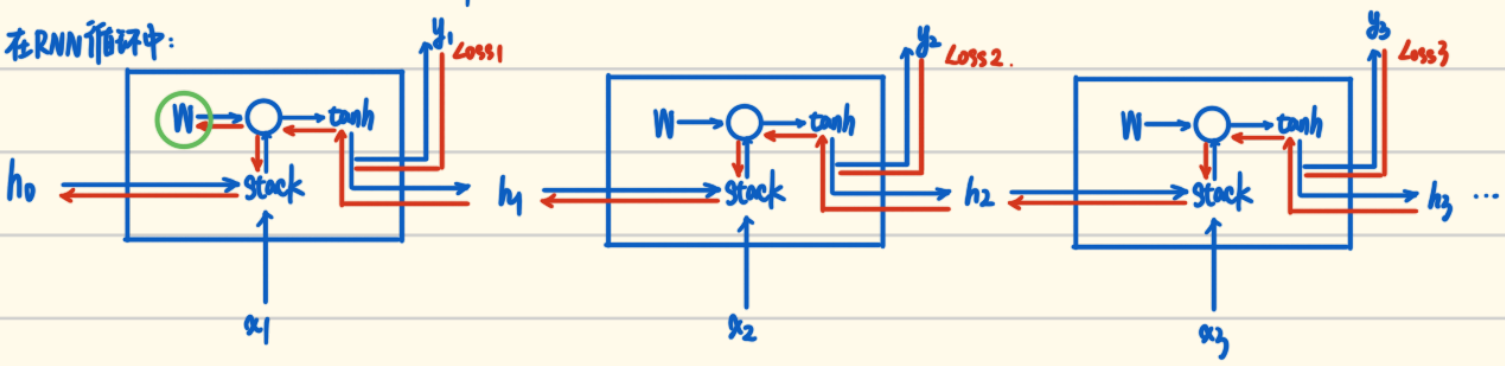


$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$
$$= \tanh\left((W_{hh}\ W_{xh})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

Backpropagation:

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}.$$

在RNN循环中:



$$\begin{cases} \frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W} \\ \frac{\partial L_t}{\partial W} = \frac{\partial L_T}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_{t-1}} \cdot \cdots \cdot \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}\right) \frac{\partial h_1}{\partial W} \end{cases}$$

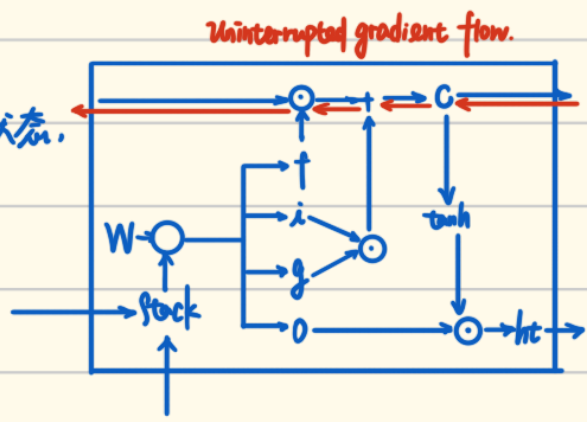
<1. Loss的梯度影响随step指数下降.

LSTM: "gate".

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$\begin{cases} C_t = f \odot C_{t-1} + i \odot g \\ h_t = o \odot \tanh(C_t) \end{cases}$$

计算4个不同的值，利用无tanh的C_t来传递隐藏状态，用$o \odot \tanh(C_t)$来输出.



Modern RNNs：状态空间模型.

- 无限上下文长度
- 计算随序列长度线性缩放.（transformers：二次扩展）.